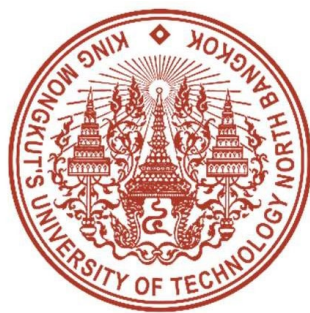




โครงการ

Object-oriented Programming

QuizApp Website (Full-stack Knowledge Quiz)

จัดทำโดย

6804062612153 นาย เมธาสิทธิ์ วิวัฒน์วรพงศ์

เสนอ

ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

โครงการ QuizApp Website (Full-stack Knowledge Quiz)

จัดทำโดย 6804062612153 นาย เมธาสิทธิ์ วิวัฒน์วรพงศ์

เสนอ ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

วิชา 040613204 Object-oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569 ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เกี่ยวกับโครงการ ชื่อโปรเจกต์: QuizApp Website

นำเสนอโดย: นาย เมธาสิทธิ์ วิวัฒน์วรพงศ์

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

Source Code: <https://github.com/s6804062612153/JavaWebSite-Project>

WebSite App: <https://quizapp-1612.onrender.com/>

บทที่ 1 ที่มาและความสำคัญของโครงการ

โครงการนี้จัดทำขึ้นเพื่อประยุกต์ใช้ความรู้ด้านการพัฒนาเว็บแอปพลิเคชันแบบ Full-Stack โดยผสมผสานการเขียนโปรแกรมฝั่งเซิร์ฟเวอร์ (Backend)

ด้วยภาษา Java ผ่านเฟรมเวิร์ก Spring Boot และการเขียนโปรแกรมฝั่งไคลเอนต์ (Frontend) ด้วย HTML, CSS และ Vanilla JavaScript

โดยนำแนวคิดการออกแบบสถาปัตยกรรมซอฟต์แวร์แบบ RESTful API และ Object-Oriented Programming (OOP) รวมถึงการจัดการ Data Transfer Object (DTO) เพื่อความปลอดภัยของข้อมูลมาใช้ในการสร้างชิ้นงานจริง

ประเภทของโครงการ โปรแกรมเว็บแอปพลิเคชัน Full-Stack (Web Application)

ประโยชน์

- เพื่อให้ผู้ใช้งานสามารถทดสอบความรู้ในหมวดหมู่ต่างๆ (Anime & Gaming, General Knowledge) ได้อย่างสะดวกและ Interactive
- เพื่อนำความรู้จากการเขียนโปรแกรมเชิงวัตถุ (OOP) มาจัดการโครงสร้างข้อมูลคำถามและคำตอบ
- เพื่อศึกษาและใช้งาน Spring Boot Framework ในการสร้าง REST API
- เพื่อฝึกฝนการจัดการ State และ DOM Manipulation บนหน้าเว็บโดยไม่ใช้ Frontend Framework (Vanilla JS)

ขอบเขตของโครงการ ความต้องการของระบบ (Functional Requirements)

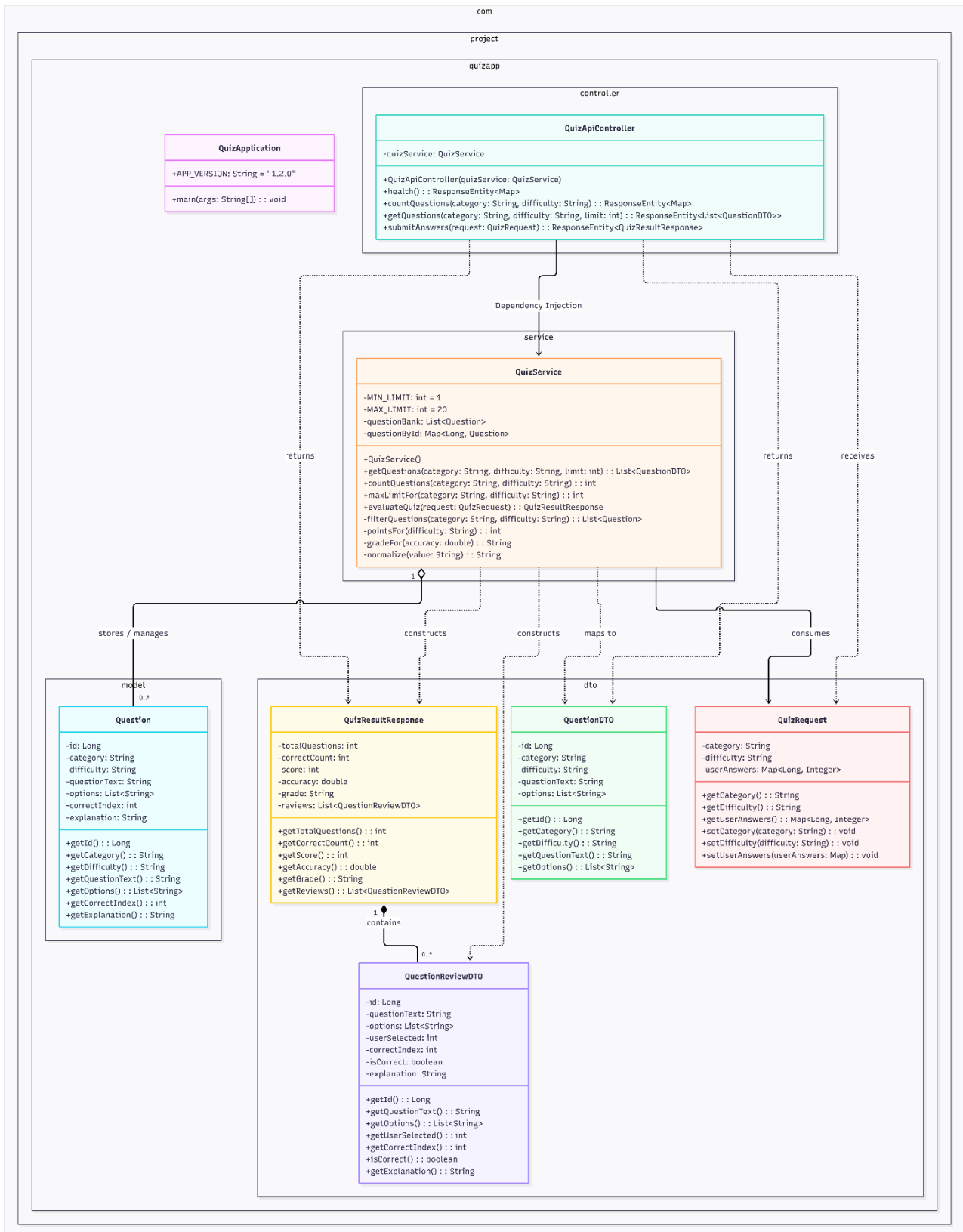
- ระบบตั้งค่าแบบทดสอบ (Quiz Setup): ผู้ใช้สามารถเลือกหมวดหมู่ (Category), ระดับความยาก (Difficulty) และจำนวนคำถามได้ โดยระบบจะคำนวณจำนวนข้อที่มีอยู่ในคลัง (API /api/quiz/count) มาจำกัดสิทธิ์ผู้ใช้แบบเรียลไทม์
- ระบบทำแบบทดสอบ (Quiz Taking):
 - มีระบบ Question Navigator แสดงสถานะข้อที่ทำแล้ว/ยังไม่ได้ทำ
 - มีระบบบันทึกสถานะชั่วคราว (Session Storage) สามารถกลับมาทำต่อได้หากปิดหน้าเว็บไป
 - รองรับการควบคุมด้วยคีย์บอร์ด (Keyboard Shortcuts)

3. **ระบบประเมินผล (Evaluation):** ระบบจะส่งคำตอบไปตรวจที่ Backend เพื่อป้องกันการโกง และตอบกลับเป็นคะแนน, ความแม่นยำ (Accuracy %), เกรด และเฉลย
4. **ระบบแสดงผลลัพธ์ (Result Visuals):** แสดงกราฟวงแหวนความแม่นยำ (SVG) และ Answer Map บอกสถานะ ถูก/ผิด/ข้าม พร้อมคำอธิบายเฉลย (Explanation) ในแต่ละข้อ
5. **UI/UX & Accessibility:** รองรับ Dark/Light Mode ตามระบบของผู้ใช้, รองรับอุปกรณ์พกพา (Responsive) และรองรับการลดแอนิเมชัน (prefers-reduced-motion)

ตารางเวลาการดำเนินโครงการ (Project Schedule)

งาน	สัปดาห์ 1	สัปดาห์ 2	สัปดาห์ 3	สัปดาห์ 4	สัปดาห์ 5
ออกแบบโครงสร้าง Class และ DTO (Backend)	x				
พัฒนา REST API (Spring Boot Controller & Service)		x			
ออกแบบ UI/UX และ Responsive CSS		x	x		
พัฒนาสคริปต์ Vanilla JS จัดการ State และเชื่อมต่อ API			x	x	
พัฒนาระบบ Session Storage (Resume Quiz)				x	
เขียน Unit Test สำหรับ QuizService					x
ตรวจสอบบั๊ก, รองรับ Dark Mode และอัปเดตเอกสาร					x

บทที่ 2 การพัฒนา



คำอธิบาย: ระบบถูกออกแบบโดยแยกส่วนการทำงานชัดเจน Controller (QuizApiController) จะรับ Request จาก Frontend และส่งต่อให้ Service (QuizService) ซึ่งจะทำหน้าที่ประมวลผลตรรกะ ดึงข้อมูลจาก Question Bank และแปลงข้อมูลเป็น DTO (Data Transfer Object) ก่อนส่งกลับไปยัง Frontend

รูปแบบการพัฒนาโครงการ (Tech Stack)

- **Backend:** Java 25 (LTS), Spring Boot 4.1.1 (Spring Web MVC)
- **Frontend:** HTML5, CSS3, Vanilla JavaScript (ไม่มี Build step)
- **Build Tool:** Maven 3.9+
- **Testing:** JUnit 5 (Spring Boot Starter Test)

แนวคิดการเขียนโปรแกรมเชิงวัตถุ (OOP Concepts Applied)

1. Data Hiding & DTO Pattern (การซ่อนข้อมูลเพื่อความปลอดภัย) เพื่อป้องกันไม่ให้ผู้ใช้แอบดูเฉยผ่าน Inspect Element บนเบราว์เซอร์
 โครงการนี้จึงใช้แนวคิด DTO โดยสร้างคลาส QuestionDTO ที่ไม่มีแอตทริบิวต์ correctIndex และ explanation

```
// src/main/java/com/project/quizapp/dto/QuestionDTO.java
public class QuestionDTO {
    private Long id;
    private String category;
    private String difficulty;
    private String questionText;
    private List<String> options;
    // ไม่มี correctIndex ถูกส่งไปที่ Frontend
}
```

2. Encapsulation (การห่อหุ้มข้อมูล) คลาสต่างๆ เช่น Question และ QuizResultResponse มีการกำหนดตัวแปรเป็น private และเข้าถึงผ่าน Getter/Setter (ผ่านการใช้ @Data ของ Lombok Library) และคลั่งคำถามถูกห่อหุ้มไว้ใน QuizService ไม่ให้คลาสอื่นเข้ามาแก้ไขได้โดยตรง

```
// src/main/java/com/project/quizapp/service/QuizService.java
@Service
public class QuizService {
    // ซ่อนคลั่งคำถามไว้เป็น private ถาวร (final)
    private final List<Question> questionBank = new ArrayList<>();
    private final Map<Long, Question> questionById = new HashMap<>();

    // ...
}
```

3. Dependency Injection & Composition QuizApiController ไม่ได้สร้าง Object ของ QuizService ขึ้นมาเอง แต่รับผ่าน Constructor ซึ่งเป็นหลักการของ Spring (IoC Container) ทำให้คลาสเชื่อมต่อกันอย่างหลวมๆ (Loosely Coupled) ง่ายต่อการทดสอบ

```
// src/main/java/com/project/quizapp/controller/QuizApiController.java
@RestController
public class QuizApiController {
    private final QuizService quizService;

    // Spring Boot จะทำ Dependency Injection ให้ผ่าน Constructor นี้
    public QuizApiController(QuizService quizService) {
        this.quizService = quizService;
    }
}
```

อัลกอริทึมที่สำคัญ (Important Algorithms)

การกรองและสุ่มคำถาม (Filtering & Shuffling ด้วย Java Stream API) การดึงคำถามตามหมวดหมู่และความยาก ใช้ Java Stream API

เพื่อความรัดกุมและมีประสิทธิภาพ รวมถึงใช้ Collections.shuffle() เพื่อให้คำถามไม่ซ้ำลำดับเดิม

```
// ใน QuizService.java
public List<QuestionDTO> getQuestions(String category, String difficulty, int limit) {
    int safeLimit = Math.max(MIN_LIMIT, Math.min(limit, MAX_LIMIT));
    List<Question> filtered = filterQuestions(category, difficulty);
    Collections.shuffle(filtered); // สุ่มลำดับคำถาม

    return filtered.stream()
        .limit(safeLimit) // จำกัดจำนวนตามที่ต้องการ
        .map(q -> new QuestionDTO(q.getId(), q.getCategory(), q.getDifficulty(), q.getQuestionText(), q.getOptions()))
        .collect(Collectors.toList());
}
```

การจัดการ State ฝั่ง Frontend ฝั่ง Frontend (Vanilla JS) ใช้ Object state ในการเก็บสถานะของแอปพลิเคชัน และซิงค์กับ sessionStorage เพื่อให้ผู้ใช้กด Refresh แล้วข้อมูลไม่หาย

```
// src/main/resources/static/js/app.js
const state = {
    questions: [],
    answers: {}, // เก็บ id คำถาม เป็น key และคำตอบเป็น value
    currentIndex: 0,
    settings: { category: 'ALL', difficulty: 'ALL', limit: 10 },
    result: null
};

// นำ state ไปบันทึกลง Browser
function saveSession() {
    sessionStorage.setItem('quizapp-session', JSON.stringify(state));
}
```

บทที่ 3 สรุป

ปัญหาที่พบระหว่างการพัฒนา

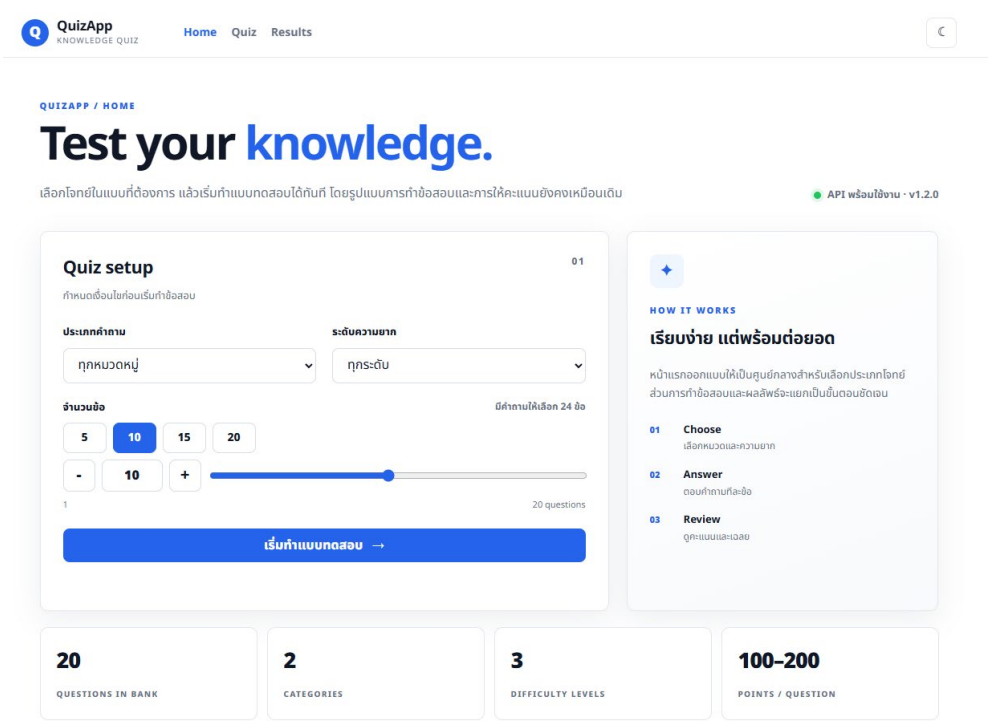
- State Management ใน Vanilla JS:** เนื่องจากไม่ได้ใช้ Framework อย่าง React หรือ Vue การจัดการ State (เช่น การเปลี่ยนข้อคำถาม การเลือกคำตอบ) จึงต้องเขียนฟังก์ชันอัปเดต DOM เอง (เช่น ฟังก์ชัน renderQuestion()) ซึ่งหากข้อมูลซับซ้อนขึ้นอาจจัดการได้ยาก โครงการนี้แก้ไขโดยจัดโครงสร้างฟังก์ชันให้แยกกันทำงานตามแต่ละ View ชัดเจน
- การรักษาความปลอดภัยของคำตอบ:** ในตอนแรกมีการส่งอบเจกต์ Question ไปยัง Frontend ทั้งหมด ทำให้สามารถกด F12 ดูเฉลยได้ จึงได้แก้ปัญหาโดยการสร้างชั้น QuestionDTO เพื่อกรองเฉลยทั้งหมดก่อนส่งให้ผู้ใช้งาน

จุดเด่นของโปรแกรม

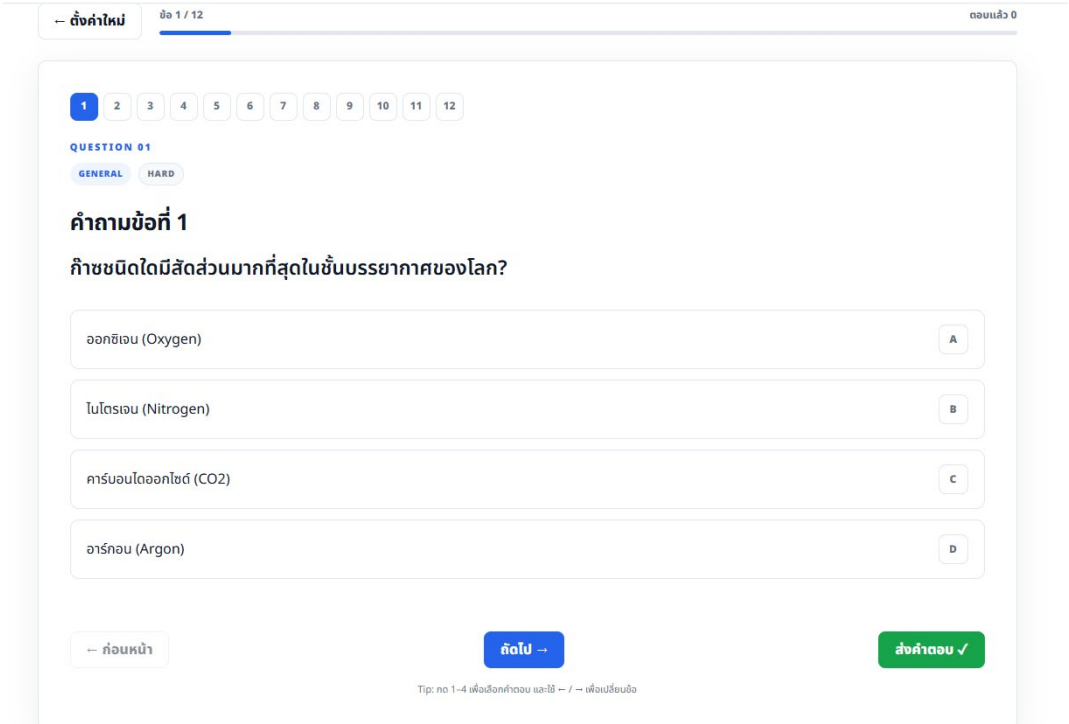
- Dynamic Question Limit:** หน้า Setup จะไปดึงข้อมูลจำนวนคำถามที่มีอยู่จริงในระบบผ่าน API /api/quiz/count ทำให้ผู้ใช้ไม่สามารถระบุจำนวนคำถามที่เกินกว่าฐานข้อมูลได้
- Session Resume:** หากผู้ใช้เผลอปิดแท็บ หรือเน็ตหลุด ระบบจะจดจำคำตอบและข้อปัจจุบันไว้ใน Session Storage เมื่อกลับมาจะมี Banner ถามว่าต้องการทำแบบทดสอบต่อหรือไม่
- Interactive Result Visuals:** วงแหวนแสดงคะแนน (Accuracy Ring) ถูกสร้างด้วย SVG และ CSS Transition ทำให้มีแอนิเมชันที่สวยงาม

รวมถึงมีหน้าต่าง Review ที่บอกอย่างละเอียดว่าผู้ใช้ตอบอะไร และที่ถูกต้องคืออะไร พร้อมคำอธิบาย

4. **Accessibility & Preferences:** เว็บรองรับ Dark/Light Mode ตามตั้งค่าของ OS ของผู้ใช้ (prefers-color-scheme) และมีตั้งค่าลดการเคลื่อนไหว (prefers-reduced-motion) สำหรับผู้ที่เวียนหัวง่าย



รูปที่ 1: หน้าจอตั้งค่าแบบทดสอบ (Setup View)



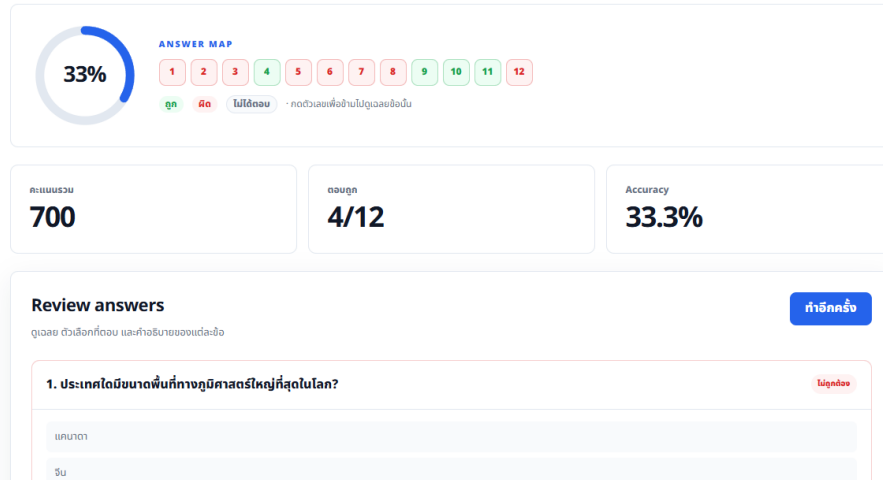
รูปที่ 2: หน้าจอระหว่างทำแบบทดสอบ (Quiz View) แสดง Question Navigator

QUIZ / RESULT

Your result.

ไม่เป็นไรครับ รอบหน้ามีโอกาสทำได้ดีกว่าเดิม

D



รูปที่ 3: หน้าจอรูปผล (Result View) แสดง SVG Accuracy Ring และ Answer Map

ข้อเสนอแนะสำหรับการพัฒนาต่อยอด (Future Work / Roadmap)

1. **เชื่อมต่อฐานข้อมูลจริง:** ในเวอร์ชันปัจจุบันคำถามถูกเก็บในรูปแบบ In-memory (ArrayList) เมื่อรีสตาร์ทเซิร์ฟเวอร์ข้อมูลใหม่จะหายไป ควรเปลี่ยนไปใช้ Database เช่น PostgreSQL หรือ MySQL ร่วมกับ Spring Data JPA
2. **ระบบ User Authentication:** เพิ่มระบบล็อกอินเพื่อเก็บสถิติประวัติการทำแบบทดสอบของผู้ใช้แต่ละคน และทำหน้าที่ Leaderboard จัดอันดับคะแนน